

Object Detection

Main points:-

-label file formats

-measuring performance

-*detection architectures* (you already ran some during your projects, you will explore some more)

The main difference from classification?

-Classification takes whole image and tells the category. You send 1 folder with say 10 subfolders. Each subfolder has images belonging to that class.

-Detection gives *bounding boxes* (I can have a model which gives circles or polygons too) inside the image along with the class name of each of the objects

The beginning of CNN based Object detection with RCNN, Fast RCNN and Faster RCNN

[R-CNN, Fast R-CNN, Faster R-CNN, YOLO — Object Detection Algorithms | Towards Data Science](#)

Then came YOLO, EfficientDet, SpineNet architectures.

[\[1912.05027\] SpineNet: Learning Scale-Permuted Backbone for Recognition and Localization](#)

[\[1911.09070\] EfficientDet: Scalable and Efficient Object Detection](#)

In the meantime you should also be aware of some general concepts related to the evaluation of these models.

Some general concepts related to object detection

- a. The label file format
- b. Measurement metrics IOU, Precision, meanAveragePrecision
- c. Output of the network

Label Formats

Sample 1.. I have 500 images, for each image I have a .txt file which has multiple entries in this format, each row depicts each object in the image

#Class-name , x-start, y-start, x-end , y-end in pixels

Car, 23 , 34 , 67 , 78

Car, 213 , 134 , 260 , 378

Dog, 13 , 134 , 160 , 278

#Some variations may include

#Class-name , x-start, y-start, width , height (in pixels)

#Class-name , y-start, x-start, y-end , x-end (in pixels)

x-start, y-start, x-end , y-end (in pixels), Class-name ,

x-start, y-start, x-end , y-end (normalized) , class-name

Ex of normalized: my image size is 320* 240 (height * width)

#Class-name , x-start, y-start, x-end , y-end in pixels

Car, 23 , 34 , 67 , 78 becomes

Car, 23/240 , 34/320 , 67/240 , 78/320

You may also have a single txt for all 500 images

#image_name, class_name, x_start,y_start, x_end,y_end

<image_name> "vvvvv"</image_name>

```
<class_name>car</car>
```

```
<x_start> 45 </xstart>
```

IOU

Let's take:

- Ground truth box: area = 100 pixels²
- Predicted box: area = 80 pixels²
- Overlap area = 60 pixels²

Then:

$$\text{Union} = 100 + 80 - 60 = 120$$

$$\text{IoU} = 60/120 = 0.5$$

So the IoU = **0.5 (50%) overlap**.

Ex.

Prediction 1: True Positive (TP)

Prediction 2: False Positive (FP)

Prediction 3: True Positive (TP)

Prediction 4: False Positive (FP)

Step 1: Compute precision & recall at each prediction

- **After 1st prediction (TP)**
 - TP = 1, FP = 0
 - Precision = TP / (TP + FP) = 1/1 = **1.0**
 - Recall = TP / GT = 1/4 = **0.25**

- **After 2nd prediction (FP)**
 - $TP = 1, FP = 1$
 - $Precision = 1/2 = \mathbf{0.5}$
 - $Recall = 1/4 = \mathbf{0.25}$
- **After 3rd prediction (TP)**
 - $TP = 2, FP = 1$
 - $Precision = 2/3 \approx \mathbf{0.667}$
 - $Recall = 2/4 = \mathbf{0.5}$
- **After 4th prediction (FP)**
 - $TP = 2, FP = 2$
 - $Precision = 2/4 = \mathbf{0.5}$
 - $Recall = 2/4 = \mathbf{0.5}$

So the raw (**recall, precision**) points are:

(0.25, 1.0), (0.25, 0.5), (0.5, 0.667), (0.5, 0.5)

After applying this:

- At recall = 0.25 $\rightarrow$ max precision to the right = 1
- At recall = 0.5 $\rightarrow$ max precision to the right = 0.667

So the corrected curve is:

(0.25, 1), (0.5, 0.667)

Step 3: Compute area under the PR curve (AP)

We integrate (sum of rectangle widths $\times$ height):

- From recall = 0.0 $\rightarrow$ 0.25, precision = 1
Area = $(0.25 - 0.0) \times 1 = 0.25$

- From recall = 0.25 $\rightarrow$ 0.5, precision = 0.667
Area = $(0.5 - 0.25) \times 0.667 = 0.1667$

Total area = $0.25 + 0.1667 = \mathbf{0.4167}$ (average precision)

11 point style

As above the raw (recall, precision) points are:

(0.25, 1.0), (0.25, 0.5), (0.5, 0.667), (0.5, 0.5)

$R = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]$

For each recall level r take $p_interp(r) = \max$ precision over all points with recall $\geq r$.

$r = 0.0$: points with recall ≥ 0.0 are all four points $\rightarrow$ precisions = {1.000, 0.500, 0.6666667, 0.500} $\rightarrow \mathbf{\max = 1.000}$

$r = 0.1$: same as above $\rightarrow \mathbf{1.000}$

$r = 0.2$: recall $0.25 \geq 0.2$ so same set $\rightarrow \mathbf{1.000}$

$r = 0.3$: only points with recall ≥ 0.3 are those with recall 0.50 $\rightarrow$ precisions = {0.6666667, 0.500} $\rightarrow \mathbf{\max = 0.6666667 (= 2/3)}$

$r = 0.4$: same as $r=0.3 \rightarrow \mathbf{0.6666667}$

$r = 0.5$: recall ≥ 0.5 includes recall=0.5 $\rightarrow \mathbf{0.6666667}$

$r = 0.6$: no point with recall $\geq 0.6 \rightarrow \mathbf{0}$

$r = 0.7$: $\mathbf{0}$

$r = 0.8$: $\mathbf{0}$

$r = 0.9$: 0

$r = 1.0$: 0

[1.0, 1.0, 1.0, 2/3, 2/3, 2/3, 0, 0, 0, 0, 0]

Average over 11 levels:

- $AP = 5 / 11 = 0.454545\dots$

This can also be done with 101 points.

Note that all the above examples consider IOU threshold at 50

We can have multiple thresholds:

IoU thresholds = [0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95]

AP has to be calculated with 11 points or 101 points as discussed above for each IOU threshold.

IoU Threshold	AP at that IoU
0.50	0.75
0.55	0.70
0.60	0.66
0.65	0.62
0.70	0.58
0.75	0.50
0.80	0.42
0.85	0.35
0.90	0.25

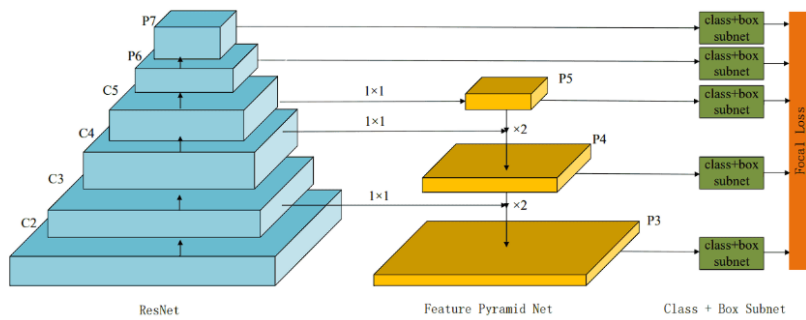
IoU Threshold	AP at that IoU
0.95	0.10
Mean (mAP@[.50:.95]) 0.493 = 49.3%	

Object Detection in Details

What you need to know:

- a) Concept of RCNN, Fast RCNN and Faster RCNN. (Go through the the article Towards Data Science)
- b) Concept of mAP computation (Go through the example)
- c) Concept of labels (Go through the sample shared)
- d) The concept of feature maps with pyramid structures
([How RetinaNet works? | ArcGIS API for Python | Esri Developer](#)) Figure 2.
- e) How feature pyramids are used for different architectures (ex. Yolo, Spinenet, EfficientDet)
[1912.05027](#) (spinenet diagram explanation)
- f) The concept of anchor boxes
[14.4. Anchor Boxes — Dive into Deep Learning 1.0.3 documentation](#)
[14.5. Multiscale Object Detection — Dive into Deep Learning 1.0.3 documentation](#)
[Non Max Suppression \(NMS\). What is Non Max Suppression, and why is... | by Vineeth S Subramanyam | Analytics Vidhya | Medium](#)
- g) How the overall system works.
Note that the anchor box concept of Retinanet will give you an overall idea of OD systems.

RetinaNet Architecture



Refer to **fig 3** in [How RetinaNet works? | ArcGIS API for Python | Esri Developer](#)
Refer to the section **Implementing Anchor generator** in [Object Detection with RetinaNet](#)

Lab 1 on OD:

- Implement Yolo with pretrained weights and save bounding box if class is person
- Use the saved person to train a classifier with 2 classes, attentive, distracted -use transfer learning with any pretrained classifier
- merge the two steps so that your overall code first runs yolo and detects person and then classifies the person as attentive or distracted

Points to note:

- use a small yolo version ex. Yolov10s
- when you save images running yolo, do pose variations with different friends. Make sure to collect minimum 450-500 images.
- yolo is trained with coco dataset. Check the label file of coco to check the class number.
- output of detection model for a image with 25 objects detected

Class number 25*1

Bounding box 25*4

Score 25*1

- Open mapcoco.txt and check class number for person...manually
- Get yolov10s (may be) running...modify the predict part, so that you can pass i)camera feed ii) check the output for class number and bounding box iii)if class number matches, you will save the bounding box
- Train classifier for your sorted images
- Combine the detection classification pipeline

Lab 2 on OD

- Use a standard Detection Model, say Yolov11 to detect person, car, dog , etc
- You should be using continuous camera feed
- In your image, check the detection classes and give the user for removal option

For example: if you have people and a dog in your picture, ask the user if he wants the dog removed.

if you have a scene with very small people, ask if you want the person removed

Implementation:-

- Check the size of the detection w.r.t your image size and consider say 20% to be removal criteria
- When the user wants the object removed, use the object bounding box and fill the box with pixels around the box.

Bb: x_1, y_1, x_2, y_2

$I[y_1:y_2, x_1:x_2, :] = \dots \text{avg_patch.}$

$h = y_2 - y_1$

$w = x_2 - x_1$

An example $h * w$ patch on the left of x_1, y_1

$\text{Sam1} = I [y_1:y_1+h , x_1-w:x_1, :]$

An example $h * w$ patch on the right of x_2, y_1

$\text{Sam2} = I [y_1:y_1+h , x_2:x_2+w, :]$